

JSX

```
let PersonCard = <div className="card">
  <h1 className="card-header">{person.getName()}</h1>
  <h2 className="card-body">Age: {person.age}</h2>
</div>
```

```
module.exports = {
  presets: ['@babel/preset-env', '@babel/preset-react']
}
```

```
{tag: 'div', props: {}}
  props:
    children: Array(2)
      0:
        props: {className: 'card-header', children: Array(1)}
        tag: 'h1'
      1:
        props:
          children: (2) ['Age: ', 24]
          className: "card-body"
          [[Prototype]]: Object
        tag: 'h2'
        length: 2
      [[Prototype]]: Array(0)
    className: "card"
    [[Prototype]]: Object
  tag: "div"
```

```
let React = {
  // should convert JSX to JS object, which can be handled by engine
  createElement:(tag, props, ...children) => {
    var element = {tag, props: {...props, children}};
    console.log(element);
    return element;
  }
}
```

2 → RENDERER

- [react-dom](#) → WEB
- [proton-native](#) → DESKTOP
- [react-native](#) → MOBILE
- [react-tv](#) → TV

index.html

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

```
import { createRoot } from 'react-dom/client'
// import './index.css' // css-loader
import App from './App.jsx' // ESM module system babel-loader

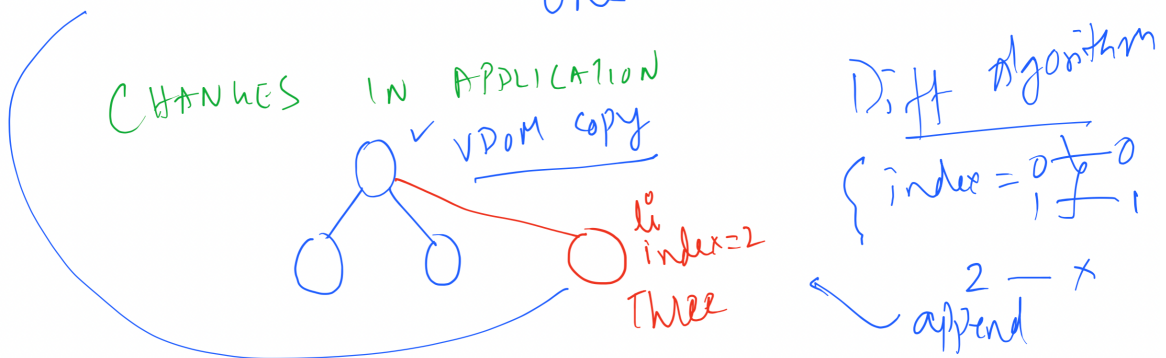
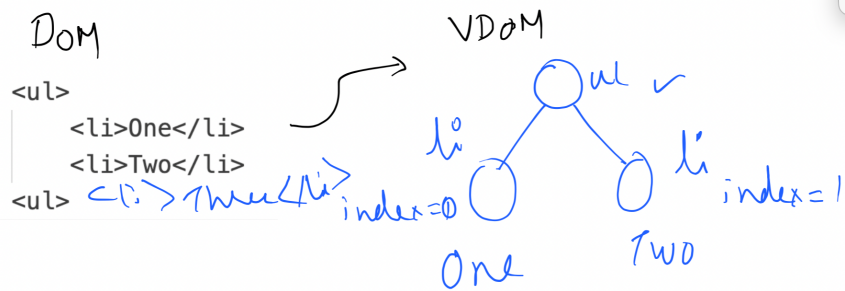
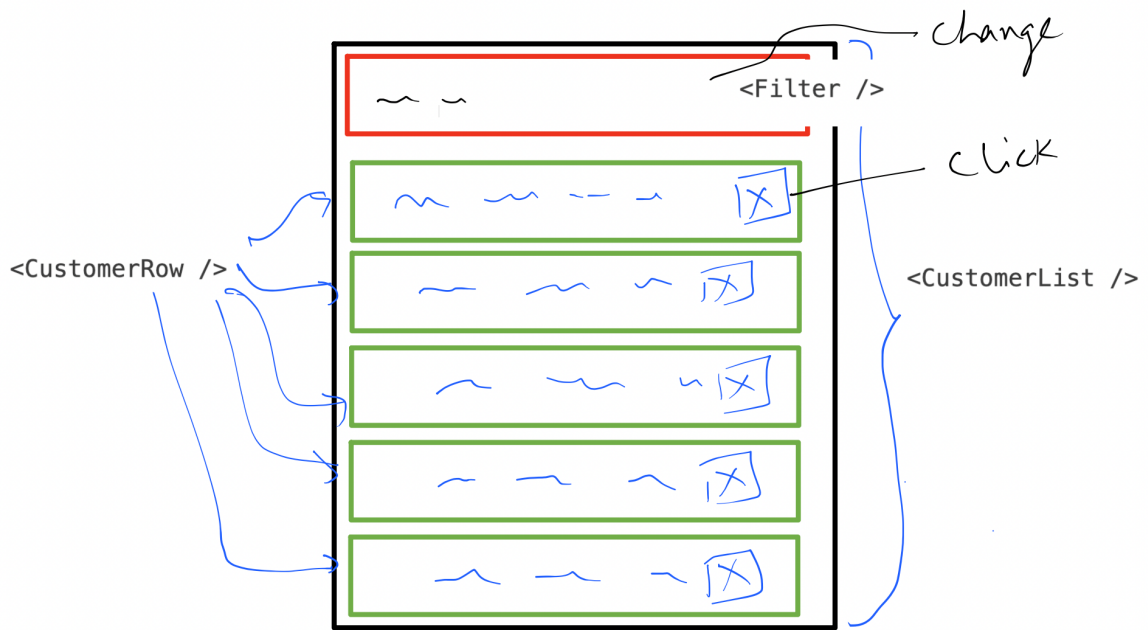
createRoot(document.getElementById('root')).render(<App />)
```

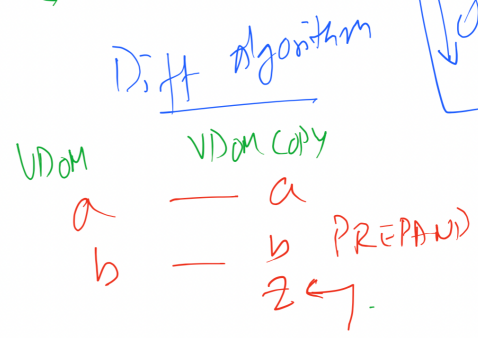
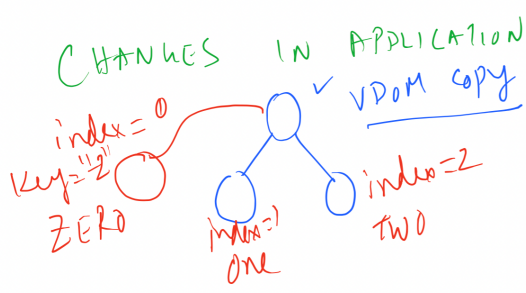
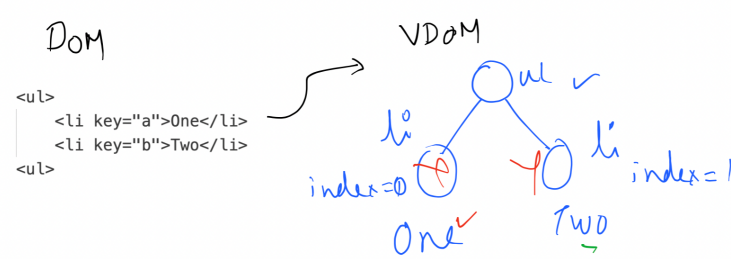
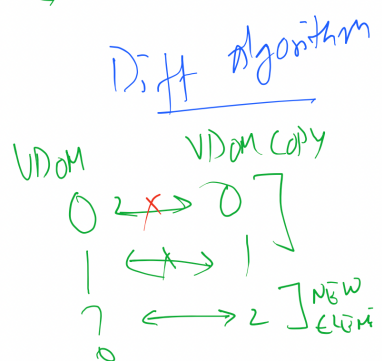
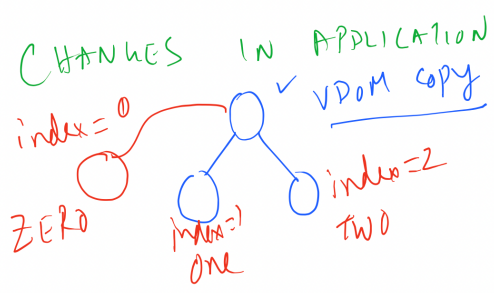
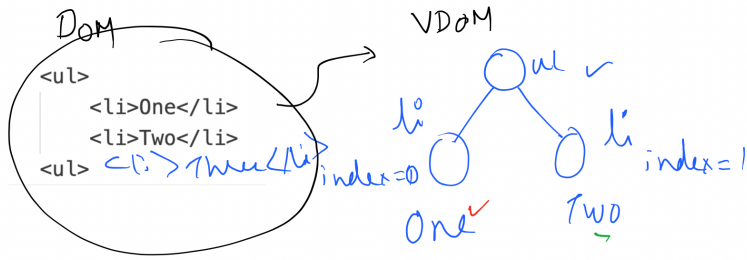
```
// functional component returns JSX
function App() {
  return (
    <div>
      <h1>React Basic Application!!!</h1>
    </div>
  )
}

export default App
```

localhost:3000

React Basic Application!!!





```

export default class CustomerList extends Component {
  name = "George"; // state in OOP, avoid this is react, because it's not considered for reconciliation
  age = 24; // state in OOP
  state = {
    // 5
  };
  deleteCustomer(id) {
    let custs = this.state.customers.filter(c => c.id !== id);
    // wrong way
    // this.state.customers = custs; // state is updated, but no reconciliation
    // right way, update state and trigger reconciliation
    this.setState({
      customers: custs
    });
  }
  render() {
    return (
      <div>
        <Filter />
        <div>
          Name: {this.name} <br />
        </div>
        {
          this.state.customers.map(cust => <CustomerRow
            deleteEvent={this.deleteCustomer(id)}
            customer={cust} key={cust.id} />)
        }
      </div>
    );
  }
}

```

```

export default class CustomerRow extends Component {
  render() {
    // destructuring, extract to local variables
    let {id, firstName, lastName} = this.props.customer;
    return (
      <div>
        {firstName} &nbsp; {lastName} &nbsp;
        <button type='button' onClick={() => this.props.deleteEvent(id)}>Delete</button>
      </div>
    );
  }
}

```

Props

```

export default class CustomerList extends Component {
  name = "George"; // state in OOP, avoid this is react, because it's not considered for reconciliation
  age = 24; // state in OOP
  state = {
    // 3
    // 5
    deleteCustomer(id) {
      let custs = this.state.customers.filter(c => c.id !== id);
      // wrong way
      // this.state.customers = custs; // state is updated, but no reconciliation
      // right way, update state and trigger reconciliation
      this.setState({
        customers: custs
      })
    }

    render() {
      return (
        <div>
          <Filter />
          <div>
            Name: {this.name} <br />
          </div>
          {
            // 2
            this.state.customers.map(cust => <CustomerRow
              delEvent={ (id) => this.deleteCustomer(id)}
              customer={cust} key={cust.id}/>)
          }
        </div>
      )
    }
  }
}

```

```

export default class CustomerRow extends Component {
  render() {
    // destructuring, extract to local variables
    let {id, firstName, lastName} = this.props.customer;
    return (
      <div>
        {firstName} &nbsp; {lastName} &nbsp;
        <button type='button' onClick={() => this.props.delEvent(id)}>Delete</button>
      </div>
    )
  }
}

```

CustomerList

```

filterCustomers(txt) {
  let custs = this.state.original.filter(c => c.lastName.toLowerCase()
    .indexOf(txt.toLowerCase()) >= 0)

  this.setState({
    customers: custs
  })
}

```

```

<Filter filterEvt={(txt) => this.filterCustomers(txt)}/>

```

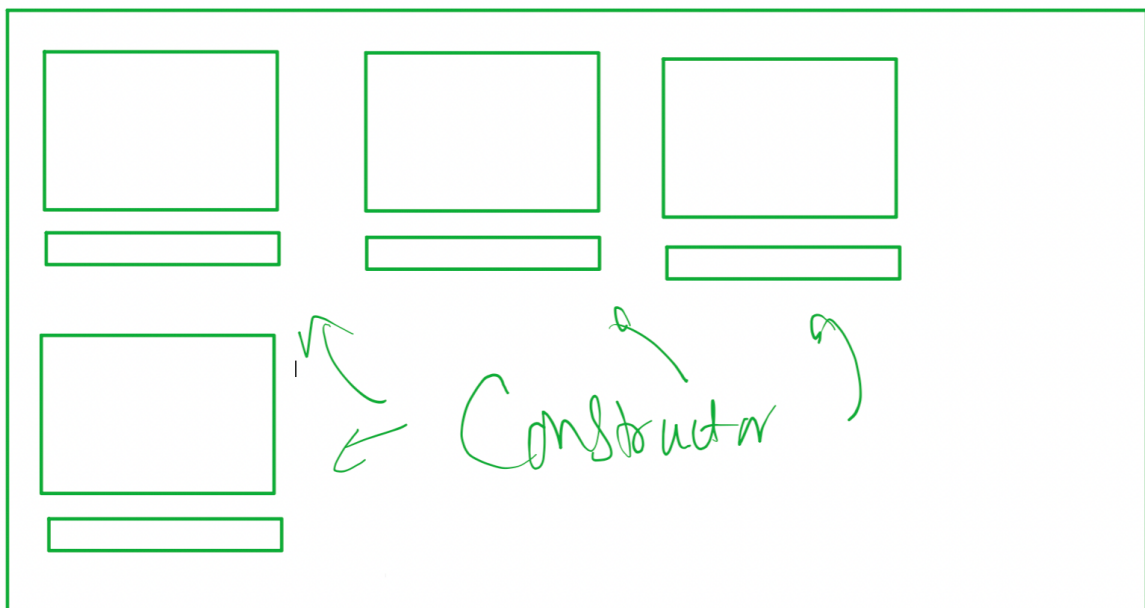
```

export default function Filter(props) {
  let {filterEvt} = props;
  return (
    <input type="text"
      placeholder="search by name"
      onChange={(evt) => filterEvt(evt.target.value)}
    />
  )
}

```

Change | Value in TextBox

Cre



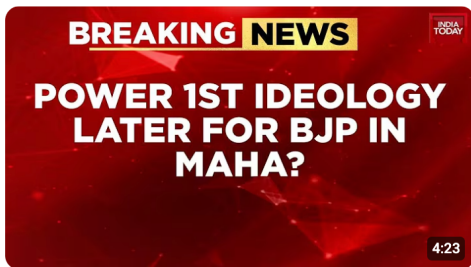
Make API call in componentDidMount() and update the state



Tata Steel Chess India 2026 | Day 1 | Live
commentary by Sagar and Amruta
ChessBase India
697 watching



Next.js Tutorial 2025 - Start Your Next.js
Journey Here
ByteGrad
47k views · 6 months ago



REACTJS, JAVASCRIPT, JAVASCRIP

<https://jsonplaceholder.typicode.com/users>

```
"id": 1,  
"name": "Leanne Graham",  
"username": "Bret",  
"email": "Sincere@april.biz",
```

```
"id": 2,  
"name": "Ervin Howell",  
"username": "Antonette",  
"email": "Shanna@melissa.tv",
```

```
"id": 3,  
"name": "Clementine Bauch",  
"username": "Samantha",  
"email": "Nathan@yesenia.net",
```

componentDidMount()

<Details />

```
"address": {  
  "street": "Victor Plains",  
  "suite": "Suite 879",  
  "city": "Wisokyburgh",  
  "zipcode": "90566-7771",  
  "geo": {  
    "lat": "-43.9509",  
    "lng": "-34.4618"  
  }  
},  
"phone": "010-692-6593 x09125",  
"website": "anastasia.net",  
"company": {  
  "name": "Deckow-Crist",  
  "catchPhrase": "Proactive didactic contingency",  
  "bs": "synergize scalable supply-chains"  
}
```

componentDidUpdate()

jsonplaceholder.typicode.com/users/2

